Отчёт по лабораторной работе №13

НКНбд-02-21

Самигуллин Эмиль Артурович

Содержание

1 Цель работы 3

2 Ход работы. 4

3 Вывод 10

1 Цель работы

Приобрести простейшие навыки разработки, анализа, тестирования и отладки приложений в ОС типа UNIX/Linux на примере создания на языке программирования C калькулятора с простейшими функциями.

2 Ход работы.

1. В домашнем каталоге создайте подкаталог ~/work/os/lab_prog.(рис. 2.1)

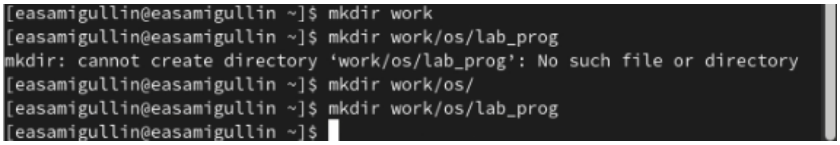

Рис. 2.1: Создал директории

2. Создайте в нём файлы: calculate.h, calculate.c, main.c. (рис. 2.2, 2.3, 2.4)

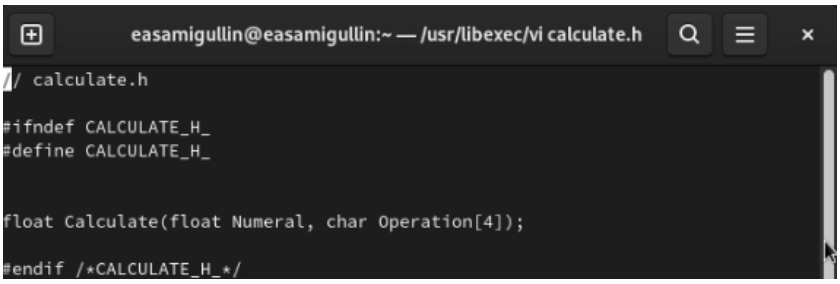

Рис. 2.2: calculate.h

```
easamigullin@easamigullin:~ — /usr/libexec/vi calculate.c
//////////////////////////////////////
// calculate.c

#include <stdio.h>
#include <math.h>
#include <string.h>
#include "calculate.h"

float
Calculate(float Numeral, char Operation[4])
{
    float SecondNumeral;
    if(strncmp(Operation, "+", 1) == 0){
        printf("Второе слагаемое: ");
        scanf("%f",&SecondNumeral);
        return(Numeral + SecondNumeral);
    }
    else if(strncmp(Operation, "-", 1) == 0){
        printf("Вычитаемое: ");
        scanf("%f",&SecondNumeral);
        return(Numeral - SecondNumeral);
"calculate.c" [dos] 52L, 1700B
```

Рис. 2.3: calculate.c

```
easamigullin@easamigullin:~ — /usr/libexec/vi main.c
//////////////////////////////////////
// main.c

#include <stdio.h>
#include "calculate.h"

int main(){
    float Numeral;
    char Operation[4];
    float Result;
    printf("Число: ");
    scanf("%f",&Numeral);
    printf("Операция (+,-,*,/,pow,sqrt,sin,cos,tan): ");
    scanf("%s",&Operation);
    Result = Calculate(Numeral, Operation);
    printf("%6.2f\n",Result);
    return 0;
}
~
~
~
"main.c" [dos] 18L, 419B
```

Рис. 2.4: main.c

3. Выполните компиляцию программы посредством gcc. (рис. 2.5)

```
easamigullin@easamigullin ~]$ gcc -c calculate.c
easamigullin@easamigullin ~]$ gcc -c main.c
easamigullin@easamigullin ~]$ gcc main.o calculate.o -o calcul -lm
easamigullin@easamigullin ~]$
```

Рис. 2.5: Компилируем с помощью gcc

4. Ошибок синтаксических не было, нечего было исправлять.

5. Создал Makefile со следующим содержанием.(рис. 2.6)

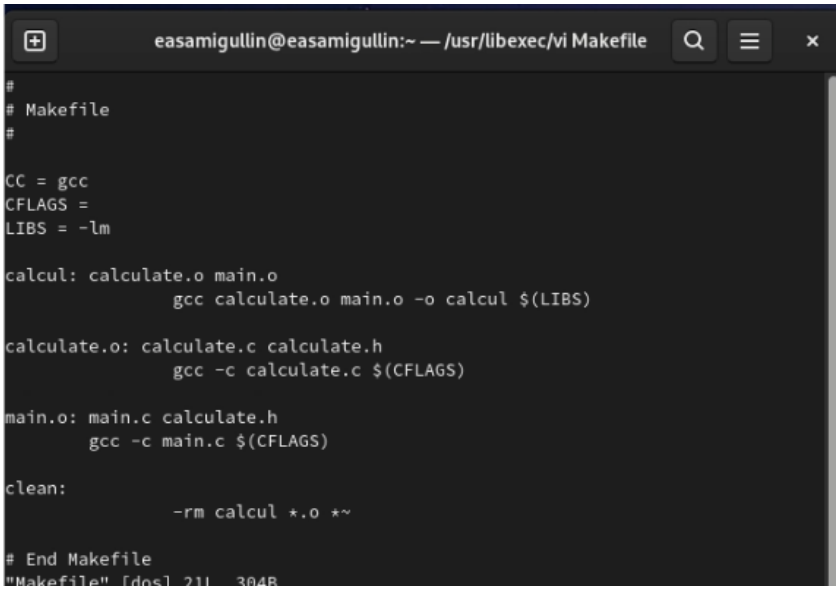

Рис. 2.6: Makefile

6. С помощью gdb выполните отладку программы calcul (перед использованием gdb исправьте Makefile)

- Запустите отладчик GDB, загрузив в него программу для отладки. (рис. 2.7)

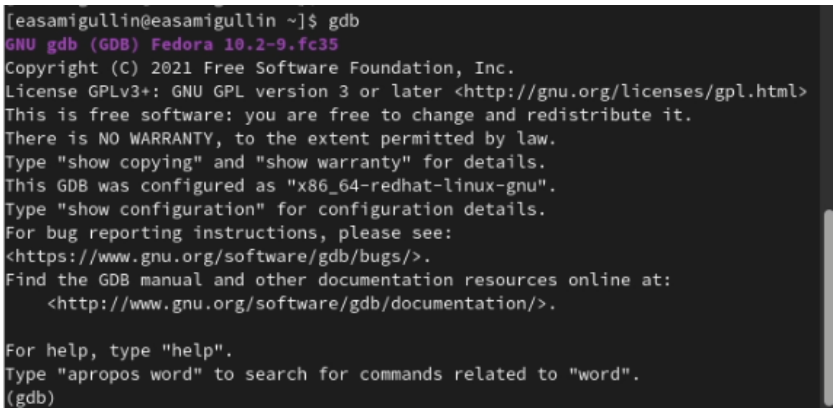

Рис. 2.7: Запуск gdb

- Для запуска программы внутри отладчика введите команду run.

- Для постраничного (по 9 строк) просмотра исходного код используйте команду list.

- Для просмотра строк с 12 по 15 основного файла используйте list с параметрами.

- Для просмотра определённых строк не основного файла используйте list с параметрами. (рис. 2.8)

```
(gdb) list
1           ////////////////////////////////////////
2           // main.c
3           #include <stdio.h>
4           #include "calculate.h"
5       int
6           main (void)
7           {
8           float Numeral;
9           char Operation[4];
10          float Result;
(gdb) list 12,15
12          scanf("%f",&Numeral);
13          printf("Операция (+,-,*,/,pow,sqrt,sin,cos,tan): ");
14          scanf("%s",&Operation);
15          Result = Calculate(Numeral, Operation);
(gdb) list calculate.c:20,29
20        scanf("%f",&SecondNumeral);
21        return(Numeral - SecondNumeral);
22      }
23      else if(strncmp(Operation, "*", 1) == 0)
24      {
25        printf("Множитель: ");
26        scanf("%f",&SecondNumeral);
27        return(Numeral * SecondNumeral);
28      }
29      else if(strncmp(Operation, "/", 1) == 0)
```

Рис. 2.8: list

- Установите точку останова в файле calculate.c на строке номер 21.
- Выведите информацию об имеющихся в проекте точка останова.
- Запустите программу внутри отладчика и убедитесь, что программа остановится в момент прохождения точки останова.
- Посмотрите, чему равно на этом этапе значение переменной Numeral.(рис. 2.9)

```
(gdb) break 21
Breakpoint 1 at 0x400868: file calculate.c, line 21.
(gdb) info breakpoints
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000000000400868 in Calculate
                                                   at calculate.c:21
(gdb) run
Starting program: /home/user/lab_prog/calcul
Missing separate debuginfos, use: yum debuginfo-install glibc-2.28-72.el8.x86_64
Число: 5
Операция (+,-,*,/,pow,sqrt,sin,cos,tan): -
Вычитаемое: 3

Breakpoint 1, Calculate (Numeral=5, Operation=0x7fffffffdfd4 "-")
    at calculate.c:21
21        return(Numeral - SecondNumeral);
(gdb) print Numeral
$1 = 5
```

Рис. 2.9: Произвел действия

- Сравните с результатом вывода на экран после использования команды.
- Уберите точки останова.(рис. 2.10)

```
(gdb) display Numeral
1: Numeral = 5
(gdb) info breakpoints
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000000000400868 in Calculate
                                                   at calculate.c:21
        breakpoint already hit 1 time
(gdb) delete 1
(gdb)
```

Рис. 2.10: Произвел действия

7. С помощью утилиты splint попробуйте проанализировать коды файлов calculate.c и main.c.(рис. 2.11,2.12)

```
Splint 3.1.2 --- 23 Jul 2021

calculate.h:6:37: Function parameter Operation declared as manifest array (siz
e
                      constant is meaningless)
  A formal parameter is declared as an array with size.  The size of the array
  is ignored in this context, since the array formal parameter is treated as a
  pointer. (Use -fixedformalarray to inhibit warning)
main.c: (in function main)
main.c:12:5: Return value (type int) ignored: scanf("%f", &Num...
  Result returned by function call is not used. If this is intended, can cast
  result to (void) to eliminate message. (Use -retvalint to inhibit warning)
main.c:14:16: Format argument 1 to scanf (%s) expects char * gets char [4] *:
                 &Operation
  Type of parameter is not consistent with corresponding code in format string
.
  (Use -formattype to inhibit warning)
   main.c:14:13: Corresponding format code
main.c:14:5: Return value (type int) ignored: scanf("%s", &Ope...

Finished checking --- 4 code warnings
```

Рис. 2.11: main.c

```
calculate.c:41:9: Return value (type int) ignored: scanf("%f", &Sec...
calculate.c:42:15: Return value type double does not match declared type floa
:
                      (pow(Numeral, SecondNumeral))
calculate.c:44:57: Return value type double does not match declared type floa
:
                      (sqrt(Numeral))
calculate.c:45:56: Return value type double does not match declared type floa
:
                      (sin(Numeral))
calculate.c:46:56: Return value type double does not match declared type floa
:
                      (cos(Numeral))
calculate.c:47:56: Return value type double does not match declared type floa
:
                      (tan(Numeral))
calculate.c:50:15: Return value type double does not match declared type floa
:
                      (HUGE_VAL)

Finished checking --- 15 code warnings
```

Рис. 2.12: calculate.c

3 Вывод

Во время выполнения лабораторной работы, мы научились работать с отладчиком GDB и узнали про splint.